

TopoBrick: Agentic Topology Sampling of Exogenous Variables for Zero-Shot Building IoT Forecasting

Xiachong Lin
dawn.lin@student.unsw.edu.au
University of New South Wales
Sydney, NSW, Australia

Du Yin
du.yin@unsw.edu.au
University of New South Wales
Sydney, NSW, Australia

Arian Prabowo
arian.prabowo@unsw.edu.au
University of New South Wales
Sydney, NSW, Australia

Hao Xue
haoxue@hkust-gz.edu.cn
Hong Kong University of Science and
Technology (Guangzhou)
Guangzhou, China

Wen Hu
wen.hu@unsw.edu.au
University of New South Wales
Sydney, NSW, Australia

Imran Razzak
imran.razzak@mbzuai.ac.ae
Mohamed bin Zayed University of
Artificial Intelligence
Abu Dhabi, United Arab Emirates

Matthew Amos
matt.amos@csiro.au
CSIRO Energy Centre
Newcastle, NSW, Australia

Sam Behrens
sam.behrens@csiro.au
CSIRO Energy Centre
Newcastle, NSW, Australia

Flora D. Salim
flora.salim@unsw.edu.au
University of New South Wales
Sydney, NSW, Australia

Abstract

Building sensors are embedded in physical topology, spatial hierarchy, and operational context, yet existing forecasters often treat them as isolated time series or rely on fixed covariate sets. We present **TopoBrick**, a training-free framework for zero-shot building IoT (Internet-of-Things) forecasting. TopoBrick uses building knowledge graphs to construct a compact structural skeleton and employs an agentic topology sampler to select target-centric exogenous variables. The selected variables are organized by deployment-time availability, separating past-known sensor states from future-known calendar, schedules, and meteorological forecasts. Across three real-world buildings, TopoBrick outperforms strong zero-shot baselines on two of the three and remains competitive with fully trained building-specific models, with the gain concentrated where the routed variables match a real physical coupling. Ablations show that topology-aware sampling is more reliable than random, ontology-only, or fixed-hop selection, especially for physically coupled HVAC and weather-driven sensing variables. The code of this work is available in <https://github.com/Dawnlxc/TopoBrick.git>

CCS Concepts

• **Information systems** → **Spatial-temporal systems**; • **Computing methodologies** → **Machine learning**; *Knowledge representation and reasoning*; • **Computer systems organization** → *Sensor networks*; • **Applied computing** → *Engineering*.

Keywords

Building IoT forecasting, building knowledge graph, zero-shot forecasting, topology-aware forecasting, Brick Schema



This work is licensed under a Creative Commons Attribution 4.0 International License. *SIGSPATIAL '26, Riverside, CA, USA*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2950-8/2026/11

<https://doi.org/10.1145/3841645.3842954>

ACM Reference Format:

Xiachong Lin, Du Yin, Arian Prabowo, Hao Xue, Wen Hu, Imran Razzak, Matthew Amos, Sam Behrens, and Flora D. Salim. 2026. TopoBrick: Agentic Topology Sampling of Exogenous Variables for Zero-Shot Building IoT Forecasting. In *The 34th ACM International Conference on Advances in Geographic Information Systems (SIGSPATIAL '26)*, November 03–06, 2026, Riverside, CA, USA. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3841645.3842954>

1 Introduction

Modern buildings are dense cyber-physical systems instrumented with hundreds to thousands of sensing, control, and operational Points. Forecasting these signals supports demand flexibility, fault detection, predictive control, and grid-interactive operation. Unlike conventional multivariate time-series benchmarks, however, a building does not expose a fixed, semantically aligned feature space. Its Points are embedded in equipment, spaces, air and water loops, and control systems, and the context relevant to a forecast is target-dependent. A zone temperature may depend on its serving equipment, setpoints, and outdoor conditions, whereas an electrical meter may depend on a different subset of the same building. Treating all available channels as interchangeable numerical inputs therefore discards the physical and operational structure that determines which signals are likely to be informative.

Recent time-series foundation models reduce deployment cost by enabling forecasting without building-specific training. However, zero-shot model transfer does not provide zero-shot context construction. In an unseen building, a forecaster still faces hundreds or thousands of heterogeneous Points whose identities, counts, and relationships differ from those in every other deployment. Channel-independent forecasting avoid this problem by independently modeling each target, while channel-dependent models assume that a useful variable set has already been specified. Even when a pre-trained forecaster can consume auxiliary variables, it does not have the ability to determine which building-specific variables are physically relevant to a given target or available at inference time. The

unresolved question is therefore: *for a target Point in an unseen building, which of the available variables should a zero-shot forecaster condition on?*

Building knowledge graphs (KG) provide a natural cross-building abstraction for this problem. The semantic schema describes Points through transferable equipment, spatial, and physical-flow relations rather than building-specific channel identities. However, a building knowledge graph, which is heterogeneous and hierarchical, is not directly a forecasting graph: most nodes represent equipment, spaces, or systems rather than time series, and relevant Points may be separated by several heterogeneous relations. In addition, KG structures are differentiated by the building blueprint and annotation preference. Learning a graph encoder would reintroduce the training and schema-alignment requirements that zero-shot deployment is intended to avoid. Our central premise is therefore that the knowledge graph should determine what the forecaster sees, rather than act as the forecaster itself: topology should serve as a transferable routing prior for constructing target-centric context.

To this end, we present **TopoBrick**, a training-free framework that uses the building knowledge graph as a routing layer between an unseen building and a frozen time-series foundation model. For each target Point, TopoBrick distills the raw KG into a compact structural skeleton and constructs a target-centric topology context. An agentic topology sampler uses this context to propose physically and operationally relevant exogenous variables. The proposals are grounded back to the graph, deterministically materialized into Point time series, and audited by a KG-grounded verifier. The selected variables are then organized by deployment-time availability: past-known sensor states are restricted to the historical window, whereas future-known calendar, schedule, and meteorological forecasts are available over both the history and prediction horizon. In this formulation, topology is responsible for constraining what information enters the forecaster without parameter updates or future-observation leakage. Across three real-world buildings, TopoBrick outperforms strong zero-shot baselines in two of the three and remains competitive with fully trained building-specific forecasters, with the largest gains occurring when the building topology exposes physically relevant exogenous variables.

The contributions of this work are summarized as follows:

- Formulate zero-shot building sensor forecasting as a topology-aware exogenous-variable selection problem over heterogeneous building knowledge graphs.
- Propose TopoBrick, a zero-shot pipeline that constructs compact building skeletons and uses an agentic topology sampler with KG-grounded verification to select target-specific exogenous variables.
- Design a deployment-faithful forecasting setup that separates past-known and future-known exogenous variables, and evaluate it on three real-world buildings against zero-shot time series foundation models and fully trained building-specific forecasters.

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 introduces the proposed methodology. Section 4 presents the experimental results. Sections 5 and 6 provide the discussion and conclusion.

2 Related Works

2.1 Forecasting with Exogenous Context

Time series forecasting has advanced rapidly with deep learning [26, 30, 31, 34], and the resulting architectures differ in ways that matter directly for this work: what they assume about the variables available alongside the target. Channel-independent (CI) methods model each series in isolation. DLinear [33] showed that a simple linear model can surpass the Transformer architectures that preceded it, including Informer [35] and its sparse attention over long sequences, redirecting the field from attention mechanisms toward representation. PatchTST [19] then demonstrated that patching the input series and treating channels independently substantially improves Transformer performance, FITS [28] and MixLinear [16] reaches comparable accuracy with a fraction of the parameters. All of these models assume the series are independent to avoid cross-variable context modeling. Channel-dependent (CD) models consider the interactions between variables, but assume the exogenous variable sets are fixed and pre-defined across channels. iTransformer [13] inverts the attention mechanism to operate across variables rather than time steps, and TimeMixer [23] mixes representations at multiple temporal resolutions. TimeXer [24] makes the distinction explicit by separating endogenous from exogenous inputs, but the exogenous set is supplied by the user rather than derived. A parallel line adapts language models to the modality by reusing a frozen backbone [14, 36], reprogramming the series into text [8, 29], or extending to multimodal inputs [7]. All of these require task-specific training.

Time series foundation models remove the task-specific training requirement. Chronos [1, 2] tokenizes time series and trains on large-scale corpora to enable zero-shot transfer. TimesFM [4] adopts a decoder-only architecture pretrained on diverse datasets, and Moirai [12, 25] unifies arbitrary frequencies and variable counts, each forecasting unseen series zero-shot. Despite this progress, these methods are largely developed and evaluated on curated benchmarks where the variable set is pre-aligned, leaving two gaps unaddressed in building IoT settings. First, channel-independent and channel-dependent models offer no principled way to decide which of a building's hundreds to thousands of points should serve as exogenous variables for a given target, either ignoring cross-sensor structure or conditioning indiscriminately on all channels, thereby introducing noise and computational overhead. Second, building-specific training incurs substantial computational costs that cannot be amortized across the long tail of heterogeneous buildings encountered in deployment, especially at large IoT scales. Foundation models remove the training cost but still treat exogenous variables as a given flat set, without accounting for the spatial, physical, or control relationships that determine their relevance to a target.

2.2 Building KGs and Semantic Analytics

Building knowledge graphs provide a structured representation of building entities and their relationships. For example, Brick Schema [3] and Haystack¹, describe equipment, locations, systems, Points, and physical or operational relationships among them.

¹<https://project-haystack.org/>

Prior work has used these semantic structures to normalize heterogeneous building metadata, including automatic classification of sensor Points into Brick classes [21]. Building semantics are also combined with time series foundation models for building energy analytics [11]. For higher-level information access tasks, BuildingQA [17] and zero-shot question-answering methods [9, 10, 18] use building KGs to ground natural-language queries in building entities and relationships. These studies demonstrate the value of building KGs for semantic normalization, retrieval, and reasoning.

Building time series is separately studied in tasks such as predictions on indoor air quality [6], light sensing [5], and electricity load [27] that include privacy-preserving formulations [32] and changes in operational patterns under exceptional conditions [20]. This literature establishes the value of both forecasting and semantic context, but does not address target-specific context construction for a zero-shot forecaster over a heterogeneous building graph. Existing forecasting and KG-based approaches therefore stop on opposite sides of the same deployment problem. Forecasting models operate on time series that have already been selected, whereas building-KG methods primarily use topology for semantic annotation, retrieval, or question answering rather than for constructing a forecaster’s inputs. TopoBrick connects these two lines of work by treating the building KG as a transferable routing prior rather than as a predictor. Building topology guides the selection of physically and operationally relevant regions, KG evidence grounds the resulting choices in valid Points, and a frozen time series foundation model performs the prediction. The resulting contribution is a training-free, target-specific mechanism for constructing exogenous context across heterogeneous buildings.

3 Methodology

Overview. Figure 1 illustrates the overall pipeline for TopoBrick. Given a target Point node, the framework first constructs a compact building skeleton from the raw KG, then uses an agentic topology sampler to select physically and operationally relevant exogenous variables. The selected variables are verified against KG evidence, materialized into time series inputs, and organized according to their deployment-time availability before being passed to a frozen time series foundation model. The agent is consulted twice per target, once to select and once to audit, and every other step is a deterministic function of the graph. See Alg. 1 for pseudocode.

Notation. Let $G = (V, E)$ denote a building KG, where V is the KG node set and $E \subseteq V \times \mathcal{R} \times V$ is the set of typed relations. Denote $V_{pt} \subset V$ as the set of Point nodes associated with time series readings, $p \in V_{pt}$ indicating a target Point, $x^{(p)}$ its observed history, and $y^{(p)}$ the values to be predicted. The exogenous-variable set relevant to p is written as $\mathcal{E}(p) \subseteq V_{pt} \setminus \{p\}$. In addition, we define T, S, H as the prediction timestep, historical context length, and prediction horizon, respectively.

3.1 Agentic Topology Subgraph Sampler

The goal of the agentic topology sampler is to use the building KG as a routing layer to identify a set of exogenous variables $\mathcal{E}(p)$ related to the given target Point p .

3.1.1 Building Skeleton Construction. To support topology-aware reasoning over heterogeneous building KGs, we first pre-process the KG by normalizing relation directions into a canonical top-down form. Inverse relations are converted to their corresponding forward forms according to the Brick ontology.

We define the building skeleton as a reduced topology graph $G_s = (V_s, E_s)$, where V_s contains structural nodes used for topology traversal and E_s contains the structural relations among them:

$$V_s = \{u \in V \mid \tau(u) \in \mathcal{T}_s\},$$

$$E_s = \{(u, r, v) \in E \mid u, v \in V_s, r \in \mathcal{R}_s\}.$$

Here, $\tau(u)$ maps node u to its top-level Brick class, $\mathcal{T}_s$ is the set of structural node types, and $\mathcal{R}_s$ is the set of relations retained for topology traversal, where

$$\mathcal{T}_s = \{\text{Equipment, Location, Collection}\},$$

$$\mathcal{R}_s = \{\text{feeds, hasPart, isLocationOf}\}.$$

Nodes carrying time series readings are excluded from V_s even when their ontology types are structural. Instead, Point nodes are maintained as attachments to skeleton nodes. For a Point p , we define its set of structural anchors as

$$A(p) = \{u \in V_s \mid (u, \text{hasPoint}, p) \in E\}.$$

The skeleton therefore separates topology reasoning from Point materialization: the agent first reasons over structural nodes and subsequently expands selected anchors into time-series Points.

3.1.2 Target-centric Topology Context. For each target Point p , we construct a target-centric topology context $C(p)$ from the building skeleton. This context summarizes where the target is located in the building and which structural regions may contain useful exogenous variables. It contains three components:

Anchor Chain. The structural path upward from the target Point: the equipment or spatial container it attaches to, and the containers above that one up to the roots the skeleton reaches. This locates the target by its physical position rather than by graph distance, and it is the path along which upstream drivers are found.

Local Topology. The sideways neighborhood at each link of that chain: for every branch above the anchor, the same-class peer cohort that the branch groups. Peers grouped by a common parent typically share a physical subsystem or control loop, so a cohort can be requested as a unit without the agent traversing the full KG.

Global Context. Summarization for global context such as external meteorology monitoring, aggregated demand, or energy usage. This context may not appear in the anchor chain but is globally relevant to the given target.

The topology context is rendered as a compact textual representation for the agent. Instead of exposing raw URI strings or the full KG, the representation uses semantic node types, relation names, Point classes, and aggregate counts, avoiding building-specific naming conventions and allowing the agent to reason over building structure at the appropriate abstraction level.

3.1.3 Exogenous-variable Selection by Agent. Given the target-centric topology context $C(p)$, the agent selects exogenous variables by reasoning over both target semantics and topology. The reasoning follows three principles:

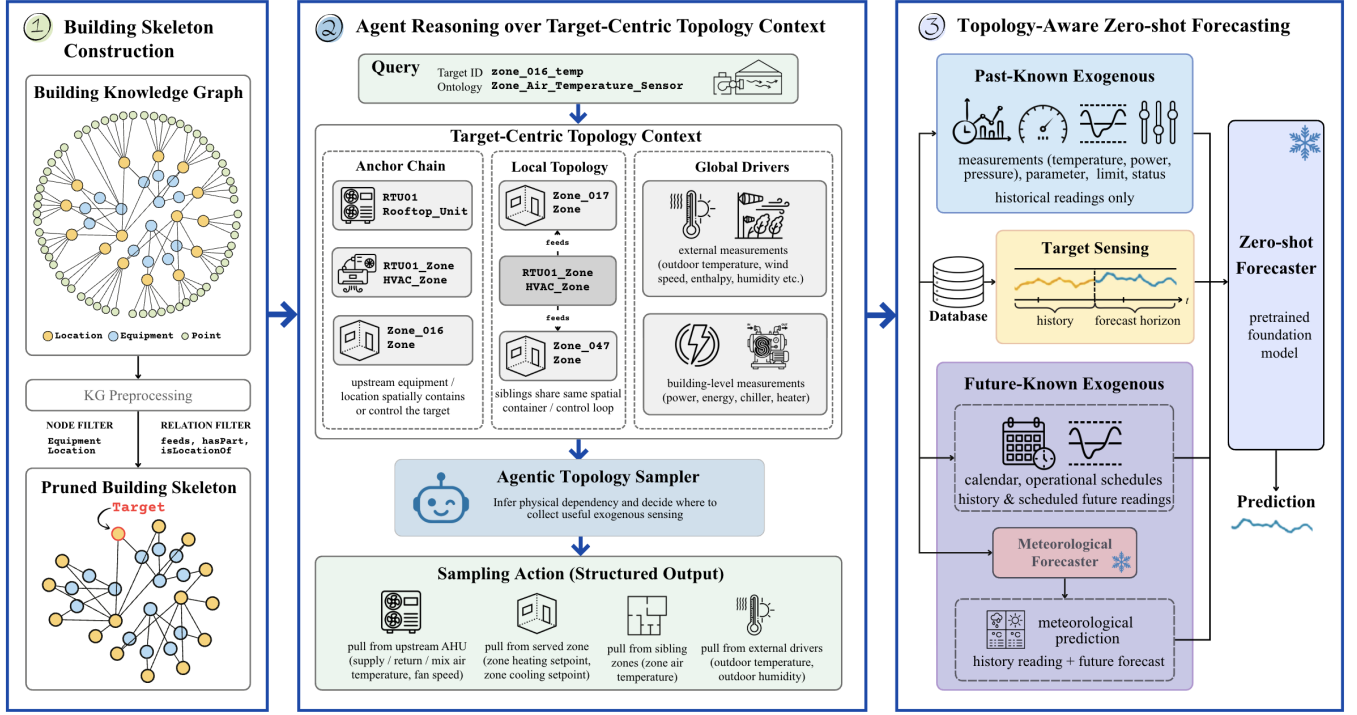


Figure 1: The visualization of the TopoBrick pipeline. (1) Building skeleton construction (2) Agentic reasoning over target-centric topology, selecting physically coupled exogenous variables (3) Topology-aware zero-shot forecaster.

Physical Relevance. The agent first identifies the physical quantity measured by the target Point and infers plausible driver categories. For example, temperature targets may depend on supply or return air temperature, outdoor conditions, setpoints, and equipment states. Pressure and airflow targets may depend on fans, dampers, valves, and loop states. Electrical targets may depend on power, current, voltage, demand, and building-level load conditions.

Topological Grounding. The agent then grounds these driver categories in the skeleton. Rather than retrieving Points solely by ontology class or fixed graph distance, it selects variables from structurally meaningful regions, such as the target anchor, upstream equipment, served zones, sibling components, or building-level hosts.

Controlled Expansion. Finally, the agent determines how broadly each structural region should be expanded. Local variables are retrieved from the target anchor or nearby equipment, whereas global variables are retrieved only when the corresponding physical quantity is building-wide by nature, such as weather or aggregate demand. This avoids uncontrolled expansion from high-degree containers. The agent outputs a structured set of sampling actions $\mathcal{A}(p) = \{(a_i, \omega_i, q_i)\}_{i=1}^m$, where $a_i \in V_s$ is a skeleton anchor drawn from $C(p)$, q_i is the requested Point class, and ω_i is one of four expansion scopes: the anchor itself, the subtree the anchor contains, the whole building for a class that is building-global by nature, or, when the target has no skeleton anchor, the cluster of Points recovered around it. These actions provide an auditable interface between agent reasoning and deterministic retrieval.

3.2 Physics Consistency Verifier

3.2.1 Exogenous-variable Materialization. The actions produced by the sampler are executed by a deterministic materialization module. For each action $(a_i, \omega_i, q_i) \in \mathcal{A}(p)$, the module resolves the anchor a_i against the anchor table constructed together with $C(p)$, expands the corresponding skeleton region under the scope ω_i , retrieves the attached Point nodes whose Brick class matches q_i , removes the target Point itself, and deduplicates repeated selections. This yields the candidate set $\mathcal{L}(p) = \text{Expand}(\mathcal{A}(p))$, where Expand collects the Points every action resolves to.

Resolution is a table lookup rather than a generation step, which places structural validity outside the agent’s reach. An anchor that does not appear in $C(p)$ has no entry, and a Brick class with no instance under the requested scope returns nothing. The agent selects among regions the knowledge graph exposes, and the expansion of a region is fixed by G_s rather than by the agent.

3.2.2 KG-grounded Verification. A second agent audits the composition of $\mathcal{L}(p)$ against the physics of the target. It receives the topology context $C(p)$ together with the Brick classes present in $\mathcal{L}(p)$ and their counts, and returns a set of classes to remove and a set of additional actions to execute:

$$(\mathcal{D}, \mathcal{A}^+) = \text{Verify}(C(p), \text{classes}(\mathcal{L}(p))).$$

Table 1 states the rubric: whether the target’s same-class spatial peers are represented, whether the physical drivers of the target quantity are present, and whether any class lacks a plausible coupling path to the target or floods the set with a single ontology.

Table 1: Rubric used by the physics-consistency verifier.

Dimension	Criterion
Peer cohort	Checks whether same-ontology spatial peers from the target’s serving or nearby equipment are included when available.
Domain drivers	Checks whether physically meaningful drivers of the target quantity are present.
Noise control	Flags off-domain variables without a plausible coupling path and overly broad same-class dumps that should be trimmed.

Removals are applied before additions, and the additional actions are materialized against the retained set, so a class the audit removes cannot re-enter through its own additions:

$$\mathcal{E}(p) = \{ \ell \in \mathcal{L}(p) \mid \text{class}(\ell) \notin \mathcal{D} \} \cup \text{Expand}(\mathcal{A}^+),$$

truncated at $n_{\max} = 60$ variables. Auditing classes rather than individual Points keeps the audit at the level at which building physics is stated: that a discharge air temperature is driven by its upstream coil and valve determines which ontologies belong in the set, not which of several interchangeable peer sensors is retained. When $\mathcal{E}(p)$ is instantiated as time series, its Points are intersected with the sensor-level usability audit of Appendix A, so a routed variable is never a sensor the quality audit has retired. Algorithm 1 in Appendix C states the two stages end to end. Section 3.3 defines how $\mathcal{E}(p)$ is organized by deployment-time availability.

3.3 Zero-shot Forecasting with Topology-aware Exogenous Variables

TopoBrick then formats the selected exogenous variables as inputs to a frozen zero-shot forecaster based on their availability at inference time.

3.3.1 Exogenous-variable Availability. The exogenous variables are divided into *past-known* and *future-known* exogenous variables.

- *Past-known exogenous variables:* Including the observations of sensor measurements and equipment states, such as power demand, flow rates, pressures, valve or damper positions, operating modes, and status that are observed up to time T , providing historical context but are masked over the prediction horizon.
- *Future-known exogenous variables:* Variables that are available over both the historical window and the prediction horizon, including
 - *Calendar features:* Deterministic functions of timestamps.
 - *Operational schedules:* HVAC schedules or setpoint programs, which are known in advance from the building management system.
 - *Meteorological Forecasts:* Observed historically and forecast over the horizon by a separate meteorological forecaster (see Section 3.3.2 for details).

3.3.2 Meteorological Forecaster. Meteorology is an important future-known signal for building IoT forecasting, but ground-truth observations are only available up to the prediction time T . Therefore,

Table 2: The buildings used for experiments. $G = (V, E)$ is the raw building KG and $G_s = (V_s, E_s)$ the skeleton derived from it. Each building is split by natural calendar for train/val/test.

	$ V / E $	$ V_s / E_s $	Train / Val / Test
LBNL59	402/499	134/225	May–Jul/Aug/Sep–Oct 2020
BTS-B	1,115/1,368	263/470	Nov–Jan/Feb/Mar–Apr 2021–22
BTS-C	11,905/10,738	1,445/2,537	Nov–Jan/Feb/Mar–Apr 2022–23

we introduce a meteorological forecaster that provides meteorological forecasts over the prediction horizon. Denote the historical meteorological readings as $\mathcal{M}_{T-S+1:T} \in \mathbb{R}^{S \times d_m}$, where d_m is the number of meteorological variables. The meteorological forecaster maps the given history to future predictions $\hat{\mathcal{M}}_{T+1:T+H} \in \mathbb{R}^{H \times d_m}$, written as

$$\hat{\mathcal{M}}_{T+1:T+H} = \mathcal{F}_m(\mathcal{M}_{T-S+1:T}).$$

The produced predictions are therefore concatenated with other future-known variables to form the future-known exogenous variable set when relevant.

3.3.3 Topology-aware Zero-shot Forecasting. Given a target Point p and its history observations $x_{T-S+1:T}^{(p)} \in \mathbb{R}^S$, the prediction task is aimed at forecasting the future sequence of p , written as $\hat{y}_{T+1:T+H}^{(p)} \in \mathbb{R}^H$. The selected exogenous variable $\mathcal{E}(p)$ for given p is converted into two aligned matrices following Section 3.3.1: past-known and future-known exogenous variables, which are written as

$$X_{T-S+1:T}^{\text{past},(p)} \in \mathbb{R}^{S \times d_p}, \quad X_{T-S+1:T+H}^{\text{future},(p)} \in \mathbb{R}^{(S+H) \times d_f},$$

respectively. d_p and d_f indicate the numbers of past-known and future-known exogenous variables selected for p . TopoBrick then performs zero-shot forecasting using a frozen pretrained forecaster $\mathcal{F}_\theta$, written as

$$\hat{y}_{T+1:T+H}^{(p)} = \mathcal{F}_\theta \left(x_{T-S+1:T}^{(p)}, X_{T-S+1:T}^{\text{past},(p)}, X_{T-S+1:T+H}^{\text{future},(p)} \right),$$

which ensures that the only building-specific computation is on metadata inference when the building is freshly registered.

4 Experiments

Datasets. We evaluate on comprehensive building operational data spanning three real-world buildings: one campus building *LBNL59* from UC Berkeley, USA [15], and two institutional buildings from Australia, *BTS-B* and *BTS-C* [22]. Each building is accompanied by a building knowledge graph that describes the semantic connections within the building, together with sensor readings collected from the installed IoT sensors. Training, validation, and testing sets are separated based on the natural calendar (see Table 2). Following the preprocessing procedure in Appendix A, 103 points in *LBNL59*, 57 points in *BTS-B*, and 619 points in *BTS-C* are selected as evaluated forecasting targets. Data granularity is processed to 15-min, where historical length $S = 96$, and prediction horizon $H = \{24, 48, 72, 96\}$, representing 6h, 12h, 18h, 24h ahead multi-step forecasting.

Baselines. We compare against three groups of baselines. *Naive baselines* include Persistence that repeats the last observed value, and Seasonal Naive that repeats the value from the corresponding

timestamp in the previous seasonal cycle. *Full-shot forecasters* include channel-independent methods as DLinear [33], PatchTST [19], FITS [28], and channel-dependent methods iTransformer [13] and TimeXer [24]. *Zero-shot forecasters* include Chronos-2 [1], Moirai [12], and TimesFM [4].

Metrics. The model is evaluated in mean squared error (MSE) and mean absolute error (MAE). Since building IoT sensors can have different physical units and value ranges, we report normalized metrics for aggregate comparison across sensors. Raw metrics are additionally reported for ontology-specific analysis, where sensors are grouped by Brick class.

Settings. All supervised baselines are trained with three random seeds, and the average performance among the three are reported. As multivariate forecasters, iTransformer and TimeXer receive the full set of forecastable (See Appendix A) sensor channels within each building, whereas the remaining univariate baselines operate on individual target series. More implementation details for supervised methods are in Appendix B. Naive and zero-shot methods are deterministic and are therefore reported as single runs. All methods are evaluated on identical forecasting windows. Calendar features are default applied for all the zero-shot forecasters. Agentic reasoning in TopoBrick is performed using gpt-oss-20b. All experiments are conducted on NVIDIA H100 GPU.

4.1 Main Results

Table 3 organizes the comparison along two axes: isolating the value of the routed exogenous context by comparing with zero-shot foundation models, and measuring the performance cost of training-free routing by comparing with full-shot building-specific models. Both axes compare TopoBrick as a whole against methods that route nothing, so neither isolates the sampler from the rest of the pipeline.

Compared to zero-shot models, Chronos2 is the strongest baseline at every cell on nMAE, and TopoBrick improves on it on two of the three buildings: improving by 3.2% to 8.5% on LBNL59 and 4.6% to 6.6% on BTS-B. The gains in BTS-C are concentrated at short horizons and slightly hurt the performance in long horizons. Since both TopoBrick and Chronos-2 read the same target history through the same frozen backbone, this isolates the routed context as the source of the difference. The potential causes for this exception are discussed in Section 5.

Compared to full-shot models, PatchTST as a channel-independent model is the most consistent of the supervised baselines, taking 23 of the 24 cells within the full-shot group, and outperforming all the channel-dependent baselines. This result indicates that the breadth of input is not a substitute for selecting a set of physically grounded variables. TopoBrick stays within 7% of PatchTST on nMAE on every building, but which method wins depends on the metric. On LBNL59, TopoBrick achieves the lowest nMSE at all four horizons (0.976/1.691/2.064/2.228), outperforming the strongest baseline by 6.8% to 12.6%, while PatchTST holds a 2.0% to 2.7% edge on nMAE. In BTS-B, TopoBrick leads on nMAE at three of four horizons by 2.6% to 4.5%, trails PatchTST by 1.3% at $H=48$, and sits 6.3% to 16.5% behind on nMSE. On BTS-C, the trained models keep a clear margin on both, 4.4% to 7.0% on nMAE and 18.2% to 23.7% on nMSE.

Table 3: Normalized errors across datasets and prediction horizons. Lower is better. Best overall results are in bold, second-best underlined, and the best baseline shaded. Supervised baselines are averaged over three training seeds.

Method	H	LBNL59		BTS-B		BTS-C	
		nMAE	nMSE	nMAE	nMSE	nMAE	nMSE
Naive Methods							
Persistence	24	0.653	2.016	0.435	0.746	0.417	0.860
	48	0.961	3.907	0.698	1.237	0.621	1.584
	72	1.108	4.846	0.802	1.405	0.718	1.521
	96	1.075	4.320	0.752	1.230	0.707	1.499
SNaive	24	0.926	4.103	0.449	0.715	0.550	1.103
	48	0.922	3.847	0.447	0.660	0.560	1.212
	72	0.922	3.749	0.447	0.644	0.567	0.891
	96	0.914	3.540	0.441	0.569	0.569	0.902
Full-shot Methods							
PatchTST	24	0.466	1.117	0.309	0.413	0.305	0.578
	48	0.623	1.820	0.383	0.478	0.399	0.813
	72	0.710	2.242	0.419	0.524	0.451	0.582
	96	0.751	2.390	0.429	0.467	0.481	0.643
FITS	24	0.588	1.704	0.396	0.487	0.386	0.634
	48	0.708	2.295	0.452	0.552	0.454	0.899
	72	0.775	2.628	0.505	0.620	0.490	0.648
	96	0.798	2.664	0.550	0.618	0.508	0.692
DLinear	24	0.493	1.250	0.320	0.425	0.336	0.578
	48	0.639	1.970	0.396	0.495	0.420	0.824
	72	0.720	2.366	0.423	0.531	0.464	0.604
	96	0.758	2.474	0.429	0.468	0.488	0.655
iTransformer	24	0.483	1.237	0.321	0.470	0.314	0.547
	48	0.629	1.947	0.403	0.522	0.409	0.837
	72	0.725	2.397	0.431	0.555	0.468	0.612
	96	0.769	2.522	0.442	0.495	0.493	0.670
TimeXer	24	0.486	1.279	0.315	0.430	0.312	0.555
	48	0.626	1.937	0.404	0.504	0.407	0.816
	72	0.719	2.495	0.428	0.528	0.457	0.587
	96	0.753	2.513	0.437	0.467	0.487	0.659
Zero-shot Methods							
Moirai2.0	24	0.578	1.551	0.391	0.663	0.385	0.746
	48	0.795	2.487	0.576	0.942	0.544	1.330
	72	0.901	2.967	0.633	0.964	0.622	1.157
	96	0.953	3.164	0.623	0.853	0.648	1.201
TimesFM2.0	24	0.546	1.543	0.360	0.558	0.359	0.648
	48	0.749	2.595	0.494	0.755	0.508	1.170
	72	0.865	3.277	0.549	0.840	0.584	1.036
	96	0.917	3.406	0.567	0.806	0.615	1.135
Chronos2	24	0.519	1.351	0.316	0.510	0.326	0.669
	48	0.688	2.220	0.409	0.628	0.427	0.965
	72	0.759	2.565	0.430	0.635	0.470	0.709
	96	0.793	2.619	0.433	0.568	0.495	0.752
TopoBrick	24	0.475	0.976	0.295	0.439	0.319	0.664
	48	0.640	1.691	0.388	0.557	0.427	0.961
	72	0.725	2.064	0.408	0.567	0.476	0.720
	96	0.768	2.228	0.413	0.505	0.502	0.779

Considering nMSE squares the residuals, where nMSE and nMAE can respond differently to a minority of large deviations, we therefore examine where each building’s nMSE is concentrated, and find that the two buildings differ in kind. On LBNL59 the concentration is temporal. The worst 5% of forecast windows account for 77.0% of the total nMSE at $H=24$ and 71.1% at $H=96$, and half of these windows fall within 14 of the 61 test days. The affected targets are not a small unreliable subset: 96 of the 103 targets contribute to the tail, and the five largest contributors account for under a quarter of it. The tail therefore corresponds to a building-wide operating condition over a limited period, which the target history alone predicts poorly and on which routed exogenous context is most informative. On BTS-B the concentration is by target. Three

sensors account for 92% of the nMSE gap to PatchTST at $H=24$, where excluding the two largest reduces the remaining gap to 1.4%, which is within the reasonable variation. On the two targets where PatchTST is weakest, TopoBrick is the more accurate of the two. The nMSE gap on this building is therefore attributable to a small number of target classes rather than to a systematically heavier error tail. Section 4.2 identifies these classes and Section 5 examines why topology provides them little useful context.

Across three real-world buildings, TopoBrick is competitive and lands within single-digit percentages of models fully trained on each building, demonstrating that topology-aware exogenous selection with a deployment-faithful organization offers a practical alternative to supervised forecasting for building IoT.

4.2 Performance Analysis by Ontology

Table 4 analyzes performance by ontology in each building. We use the Chronos-2 model where calendar features are provided as default future-known exogenous variables as the compared baseline for computing the relative raw MAE change Δ for each ontology. In addition, the number of sensors and raw MAE by ontology are also tabulated for reference. Green/red indicates improvement/degradation compared to the baseline.

On LBNL59, the improvements are concentrated on air-side temperature variables. In particular, *Supply Air Temperature Sensor* achieves consistent reductions across all horizons, from -22.8% at 24 steps to -17.7% at 96 steps. Similar but smaller improvements are observed for *Return Air Temperature Sensor*, *Mixed Air Temperature Sensor*, and *CO2 Sensor*. These results suggest that topology-aware exogenous variables are especially useful when the target signal is governed by HVAC air-loop interactions. In contrast, the improvement on *Zone Air Temperature Sensor* decreases with the prediction horizon and becomes slightly positive at 96 steps, indicating that zone-level temperature may already be well captured by temporal persistence and that additional context is less beneficial when unobserved occupancy or control actions dominate longer-term variations. BTS-B exhibits a more selective pattern. The strongest improvements are observed for *Differential Pressure Sensor*, *Discharge Water Temperature Sensor*, and *Return Water Temperature Sensor*, with large error reductions across all horizons. This indicates that the sampled context is particularly effective for water-loop and pressure-related variables in this building. However, several air-temperature and outdoor-air variables show limited or negative gains, suggesting that semantic similarity alone is insufficient and that the utility of exogenous variables depends on the underlying building topology and operating regime. BTS-C provides the most diverse ontology-level evidence. Large and stable improvements are observed for weather-coupled and thermodynamic variables, including *Outside Air Temperature Sensor*, *Outside Air Enthalpy Sensor*, *Air Enthalpy Sensor*, *Exhaust Air Temperature Sensor*, *Return Air Temperature Sensor*, and *Outside Air Dewpoint Sensor*. These classes are directly affected by outdoor conditions, air mixing, and heat exchange processes, making them well suited to topology-aware and future-context-aware modeling. By contrast, *Electrical Power*

Sensor, *Demand Sensor*, *Supply Air Static Pressure Sensor*, and *Discharge Air Flow Sensor* show increased error. These signals are often driven by stochastic equipment usage, occupancy, and closed-loop control actions, which may not be fully captured by the available exogenous variables.

The split is sharp enough to read as a rule. Where a target’s drivers are themselves instrumented and structurally reachable, routing finds them and the error falls. Where the dominant driver is occupancy, discretionary equipment use, or a supervisory control loop, no Point in the graph stands for it, and the sampler can only return variables that are structurally adjacent without being causally related.

4.3 Comparison on Sampling Strategy

We compare the proposed agentic sampler against several alternative sampling strategies to validate the information gains from the topology-aware sampler.

- *Random Sampler*: Randomly selects building Points as exogenous variables without considering either semantic alignment or topological dependency.
- *Same Ontology Sampler*: Selects points that share the same Brick ontology class with the target sensor as exogenous variables.
- *k-hop Sampler*: Selects Points within a fixed k -hop neighborhood of the target node as exogenous variables.

To isolate the effect of the sampling strategy, we evaluate all samplers under the same forecaster and compare their error changes against a baseline without considering KG. For each sampler, we follow the rule in Section 3.3.1 to separate the exogenous by availability. Oracle meteorological forecasts are applied as the future-known exogenous to remove uncertainty from external forecasting errors and ensure that the comparison focuses on the quality of exogenous variable selection. Similar to Section 4.2, calendar features are applied default. We report the percentage changes in normalized MAE and MSE, where negative values indicate improvement.

Figure 2 visualizes the experiment results, which show consistent achievement for the strongest improvements by the proposed agentic sampler. On LBNL59, it gives the largest reduction on both metrics at every horizon, reducing normalized MAE by 2.4 to 5.1 times as much as the strongest alternative and normalized MSE by 1.4 to 2.0 times. 2- and 3-hop sampling score identically here, indicating that merely using local graph neighborhoods is insufficient. Selecting structurally meaningful variables is more important than expanding the neighborhood size. The advantage of TopoBrick agentic sampler is most pronounced on BTS-B. It leads the strongest alternative at every horizon, reducing normalized MAE by 1.3 to 1.7 times as much and normalized MSE by 1.6 to 3.9 times as much. In contrast, random sampling consistently increases error, confirming that injecting arbitrary building signals can introduce noise rather than useful context. The same-ontology sampler performs competitively on BTS-B, particularly for normalized MAE, but its improvement remains weaker than TopoBrick. This suggests that semantic similarity is useful when same-class sensors share common operational patterns, but it still misses cross-ontology physical dependencies such as the physical coupling in spatial containers or control loops. On BTS-C, all samplers show smaller gains,

²The unit of *Zone Air Temperature Sensor* in LBNL59 are mixed with $^{\circ}\text{F}$ and $^{\circ}\text{C}$. We convert all $^{\circ}\text{F}$ readings to $^{\circ}\text{C}$ for convenient.

Table 4: Per-ontology performance by building: raw MAE (rMAE) and relative change $\Delta\%$ (baseline→model) at 24/48/72/96 steps.

Ontology		Horizon		24		48		72		96	
		N	Unit	rMAE	$\Delta\%$	rMAE	$\Delta\%$	rMAE	$\Delta\%$	rMAE	$\Delta\%$
LBNL59	Zone_Air_Temperature_Sensor	67	°C ²	0.184	−6.3	0.259	−3.4	0.296	−0.2	0.316	+0.8
	CO2_Sensor	11	ppm	6.237	−5.7	7.692	−5.3	8.267	−4.5	8.630	−3.6
	Building_Electrical_Meter_Sensor	5	kW	1.031	−2.6	1.240	−0.6	1.399	+0.1	1.562	−0.3
	Mixed_Air_Temperature_Sensor	4	°C	0.537	−10.1	0.798	−8.8	0.959	−6.1	1.017	−5.9
	Return_Air_Temperature_Sensor	4	°C	0.227	−10.8	0.317	−9.9	0.359	−9.6	0.370	−8.4
	Supply_Air_Temperature_Sensor	4	°C	0.243	−22.8	0.355	−21.9	0.429	−20.2	0.467	−17.7
	Chilled_Water_Temperature_Sensor	2	°C	0.701	−3.2	0.870	−1.4	0.944	−0.5	1.008	+0.1
	Outdoor_Air_Temperature_Sensor	2	°C	1.569	−2.5	2.184	−3.0	2.541	−2.3	2.753	−2.6
	Outdoor_Air_Humidity_Sensor	1	%	6.777	+0.4	10.490	+1.2	12.912	+3.7	14.038	+3.9
BTS-B	Hot_Water_Temperature_Sensor	2	°C	0.627	+1.0	0.771	+1.2	0.910	+0.7	0.944	+0.4
	Air_Temperature_Sensor	23	°C	0.549	−1.2	0.794	+1.9	0.841	+1.7	0.869	+2.7
	Discharge_Air_Temperature_Sensor	10	°C	0.922	+0.8	1.234	+2.9	1.345	+3.1	1.380	+3.1
	Filter_Differential_Pressure_Sensor	5	–	8.496	+2.9	18.804	+2.0	22.603	−0.9	19.887	−2.1
	Average_Zone_Air_Temperature_Sensor	2	°C	0.442	−3.4	0.650	−3.3	0.730	−0.6	0.724	+0.6
	Differential_Pressure_Sensor	2	–	2.272	−79.2	2.821	−78.3	3.283	−71.8	3.513	−67.4
	Discharge_Water_Temperature_Sensor	2	°C	4.651	−33.2	5.082	−36.4	4.952	−36.8	4.927	−35.0
	Return_Water_Temperature_Sensor	2	°C	3.599	−35.9	4.043	−37.2	4.280	−36.0	4.372	−32.7
	Warmest_Zone_Air_Temperature_Sensor	2	°C	0.522	−3.1	0.746	−1.8	0.802	+4.7	0.894	+7.8
BTS-C	Chilled_Water_Differential_Temperature_Sensor	1	°C	0.609	−1.5	0.785	−0.6	0.879	+0.3	0.901	−1.1
	Outside_Air_Temperature_Sensor	6	°C	1.426	+1.7	1.851	+0.7	1.987	+0.9	2.073	+1.0
	Outside_Air_Humidity_Sensor	1	%	6.896	+8.2	8.739	+4.7	9.373	+2.9	9.868	+2.9
	Water_Temperature_Sensor	1	°C	0.627	−1.6	0.675	+5.7	0.807	−1.3	0.713	+2.9
	Electrical_Power_Sensor	202	kW/W	2.909	+2.8	3.627	+6.4	3.926	+5.8	3.908	+5.1
	Zone_Air_Temperature_Sensor	67	°C	0.355	−9.8	0.534	−8.0	0.632	−4.2	0.695	−1.8
	Air_Temperature_Sensor	55	°C	0.482	−7.8	0.663	−4.5	0.726	−4.6	0.764	−4.9
	Relative_Humidity_Sensor	44	%	2.933	−8.8	4.302	−9.1	5.119	−6.9	5.608	−5.5
	Air_Flow_Sensor	26	–	41.952	+0.8	51.548	+2.7	59.933	+2.5	62.457	+2.2
BTS-D	Supply_Air_Temperature_Sensor	24	°C	0.767	−6.2	1.082	−0.5	1.241	+2.2	1.331	+3.2
	Average_Zone_Air_Temperature_Sensor	23	°C	0.354	−8.0	0.541	−3.4	0.633	+0.0	0.688	+1.2
	Supply_Air_Static_Pressure_Sensor	16	–	2.272	+4.3	2.284	+9.2	2.545	+10.6	2.728	+11.0
	Outside_Air_Temperature_Sensor	14	°C	0.655	−40.2	0.827	−41.7	0.883	−42.4	0.915	−42.5
	Air_Enthalpy_Sensor	12	–	1.187	−33.4	1.661	−36.1	1.968	−34.7	2.167	−33.7
	Return_Air_Humidity_Sensor	10	%	2.332	−13.2	3.436	−11.2	4.024	−8.7	4.399	−7.9
	Return_Air_Temperature_Sensor	10	°C	0.454	−25.9	0.608	−25.0	0.690	−22.6	0.738	−22.2
	Temperature_Sensor	10	°C	0.811	−5.4	0.960	−5.4	0.855	−3.6	0.882	−0.3
	Outside_Air_Enthalpy_Sensor	9	–	1.529	−35.7	2.133	−35.7	2.403	−35.3	2.581	−34.9
BTS-E	CO2_Sensor	9	ppm	16.511	−8.6	23.081	−4.9	25.593	−3.2	27.718	−1.2
	Outside_Air_Humidity_Sensor	8	%	4.499	−17.3	6.265	−15.3	6.996	−13.2	7.456	−11.9
	Demand_Sensor	8	kW	100.086	+10.8	104.932	+13.0	18.339	+20.6	19.013	+18.6
	Return_Air_Enthalpy_Sensor	7	–	1.329	−19.6	1.908	−21.4	2.257	−19.2	2.455	−19.0
	Exhaust_Air_Temperature_Sensor	5	°C	0.519	−48.9	0.659	−50.1	0.712	−50.7	0.742	−50.8
	Exhaust_Air_Humidity_Sensor	5	%	5.063	−12.8	7.129	−11.3	8.019	−8.4	8.493	−7.5
	Zone_Air_Humidity_Sensor	5	%	1.662	−4.6	2.508	−4.9	3.020	−3.7	3.321	−3.2
	Chilled_Water_Flow_Sensor	5	–	0.285	−4.5	0.442	−9.0	0.657	−8.4	0.733	−8.8
	Discharge_Air_Flow_Sensor	5	–	52.701	+4.5	89.937	+12.7	111.443	+15.6	123.168	+15.4
BTS-F	Supply_Air_Humidity_Sensor	4	%	4.459	−7.0	6.500	−4.2	7.506	−1.2	8.056	+0.3
	Enthalpy_Sensor	4	–	1.129	−16.5	1.749	−17.7	2.183	−16.0	2.455	−14.6
	Chilled_Water_Supply_Temperature_Sensor	4	°C	1.109	−0.3	1.396	−3.9	1.523	−6.8	1.637	−7.6
	Chilled_Water_Return_Temperature_Sensor	4	°C	1.260	−4.2	1.761	−0.7	2.051	+1.5	2.245	+4.0
	Outside_Air_Dewpoint_Sensor	3	°C	0.919	−23.0	1.213	−28.5	1.348	−29.4	1.428	−29.3
	Dewpoint_Sensor	3	°C	1.102	−11.1	1.523	−8.1	1.728	−5.6	1.820	−4.7
	Filter_Differential_Pressure_Sensor	3	–	3.444	−15.1	4.444	−27.5	4.410	−33.2	5.278	−30.7
	Humidity_Sensor	2	%	4.492	−0.3	7.117	+11.5	7.831	+12.5	8.259	+12.5
	Mixed_Air_Temperature_Sensor	2	°C	0.596	−10.9	0.786	−5.0	0.854	−4.4	0.929	−3.2
BTS-G	Hot_Water_Return_Temperature_Sensor	2	°C	1.969	−0.8	2.497	+2.7	2.390	+2.4	2.233	+3.1
	Water_Temperature_Sensor	2	°C	1.300	−0.5	2.263	+1.5	3.165	+2.1	3.923	+0.7
	Hot_Water_Supply_Temperature_Sensor	2	°C	0.867	+8.9	1.047	+11.2	1.123	+11.5	1.135	+9.8
	Chilled_Water_Differential_Pressure_Sensor	2	–	0.667	+10.2	0.666	+20.2	0.186	+11.6	0.149	+6.0
	Air_Differential_Pressure_Sensor	1	–	1.374	−3.6	1.473	−9.6	1.111	−3.3	1.195	−2.4
	Cooling_Demand_Sensor	1	kW	0.288	−1.3	0.317	+0.2	0.334	−0.9	0.343	−0.1
	Power_Sensor	1	kW	1.984	−3.5	2.022	−4.0	0.121	−10.0	0.123	−8.3

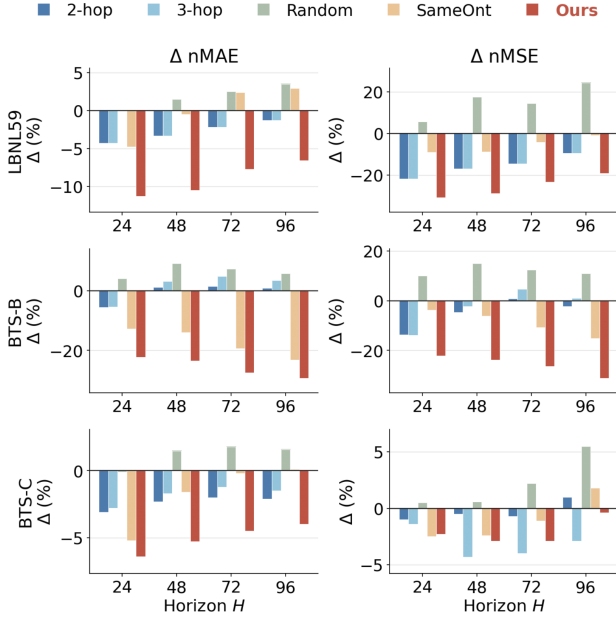


Figure 2: Visualization of the sampler comparison. The x-axis indicates the prediction horizon, and y-axis indicates the Δ against the baseline. Negative Δ means performance improvement.

suggesting that the benefit of exogenous variables is more dataset-dependent. Nevertheless, TopoBrick remains the best or among the best strategies across most horizons, indicating that topology-aware sampling provides robust benefits.

The results support the central hypothesis that building sensors should not be treated as isolated time series. Same-ontology retrieval captures semantic similarity but may select physically unrelated sensors, while fixed k -hop sampling captures local connectivity but lacks relevance filtering. TopoBrick provides a better trade-off by selecting exogenous variables that are both structurally connected and operationally meaningful, leading to more reliable forecasting improvements.

4.4 Ablations Study

4.4.1 Ablation on Future Known Exogenous Variables. We conducted an ablation study by incrementally augmenting the baseline with each covariate group. A Chronos-2 with calendar features as default future-known exogenous variables is employed as the baseline for computing the relative errors. The evaluated exogenous groups include past observations of sensor and equipment (+PastObs) which only consider past-known exogenous that defined in Section 3.3.1, future-known operational schedules (+OpSched), and the meteorological forecasts (+WeatherFcst). As the visualized results in Figure 3, the meteorological forecasts (+WeatherFcst) yield the largest and most consistent performance improvements. Compared to the baseline, the maximum nMAE decreased by 8.5% on LBNL59 and by 6.6% on BTS-B, demonstrating the critical impact of future-known meteorological factors in building sensing.

Operational schedules (+OpSched) ranked second, offering improvements of up to 7.3%, which reflect the tight coupling between HVAC commands/setpoints and observed dynamics. In contrast, past observations of sensor and equipment (+PastObs) provided minimal benefit and even had a detrimental effect in BTS-B. Our analysis suggests that because these signals are masked during the prediction horizon, enriching historical context without providing future information, zero-shot models cannot always reliably leverage them. Furthermore, the BTS-C dataset under a long horizon does not seem to respond well to our ablation modules, which will be discussed in Section 5. This experiment also suggests that the information gains are mainly from future-known exogenous factors, especially meteorological forecasts, highlighting the value of modeling formulations that account for signal availability.

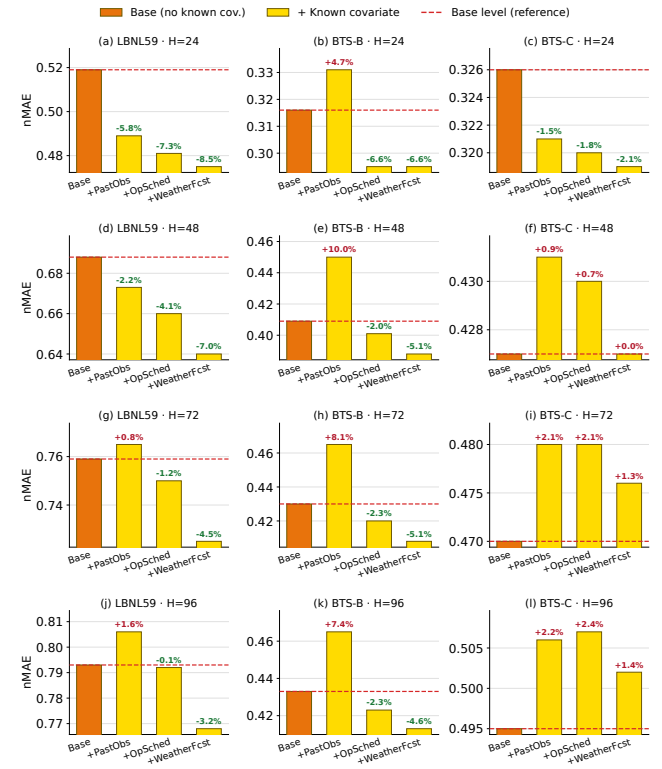


Figure 3: The average normalized MAE (nMAE) of availability-aware exogenous variables across prediction horizons. The x-axis indicates the evaluated exogenous groups, and y-axis indicates the errors under the given group, dashed indicates the baseline performance, and labels annotate the Δ against the baseline. Lower is better.

4.4.2 Sensitivity to Meteorological Forecast Quality. This experiment evaluates the performance sensitivity towards the quality of meteorological forecasts by injecting controllable Gaussian noise σ into the oracle meteorological readings as future-known exogenous variables. The smaller the σ , the higher the meteorological forecast quality, where $\sigma = 0$ is the oracle. The results in Figure 4 show

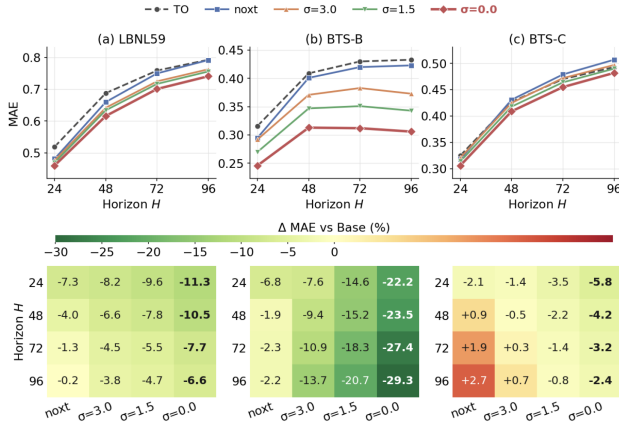


Figure 4: Sensitivity analysis of model performance to meteorological forecast quality.

a consistent monotonic trend: as meteorological forecast quality improves, forecasting error decreases. This confirms that weather is a useful future-known exogenous signal. On LBNL59, the oracle meteorological setting reduces nMAE by 11.3%, 10.5%, 7.7%, and 6.6% from 24 to 96 steps, respectively. Even with noisy meteorological inputs, the model remains better than the non-weather variant, indicating that the framework does not require perfect weather forecasts to be beneficial. On BTS-B, with the oracle meteorological inputs, the improvement increases from 22.2% at 24 steps to 29.3% at 96 steps. This suggests that meteorological conditions provide strong long-horizon explanatory power for this building. Under the noisy setting $\sigma = 3.0$, BTS-B still obtains consistent reductions across all horizons, showing that the method is robust to realistic forecast uncertainty. BTS-C shows a weaker but still informative pattern. Without meteorological forecasts (noxt), the model slightly degrades at longer horizons. Adding noisy weather already recovers part of this loss, and oracle weather further improves the results. This suggests that BTS-C is less weather-dominated and potentially more affected by unobserved occupancy, equipment usage, or closed-loop control dynamics. Nevertheless, the monotonic improvement from noisy to oracle weather confirms that meteorological quality remains relevant.

5 Discussion

In heterogeneous buildings, the available exogenous points for a given target are numerous, unevenly distributed, and partially relevant to the given target. An arbitrary selection of exogenous points can degrade performance, while fixed radius graph neighborhoods are semantically unaware. The main bottleneck is the lack of a target-specific mechanism to decide the relevance. TopoBrick uses the building KG to impose this routing prior: the graph does not directly predict the future, but constrains what the forecaster should condition on.

The results characterize the building KG primarily as a routing prior: it constrains what the forecaster conditions on rather than predicting future values directly. However, its benefits depend on

two conditions: whether the target’s dominant drivers are observable in the graph and whether the selected signals are available at prediction time. The ontology-level results in Section 4.2 reveal this pattern. Targets governed by instrumented air-side, water-side, and thermodynamic interactions generally improve, whereas electrical power, demand, airflow, and static pressure are often limited.

The first condition defines the coverage boundary of topology-based routing. The latter quantities are strongly influenced by occupancy, discretionary equipment use, and supervisory control logic, none of which is represented as a Point in the evaluated buildings. No routing strategy can recover a driver that is neither measured nor represented in the KG. In such cases, the sampler may return structurally nearby but weakly predictive variables, diluting the information already contained in the target’s history. BTS-C illustrates this limitation most clearly. Addressing it requires richer sensing and metadata coverage, rather than simply expanding the graph neighborhood or using a larger forecaster.

The second condition is deployment-time availability. Past-known sensor and equipment states enrich the historical context but provide no information within the prediction horizon, limiting their value, particularly for longer forecasts. Future-known variables, such as operational schedules and meteorological forecasts, provide more consistent gains because they remain available throughout the horizon. The weather-quality analysis further shows that improved forecasts yield larger gains, while imperfect forecasts can still be useful. Relevant context must therefore be selected jointly according to physical relevance and prediction-time availability.

The formulation may extend beyond forecasting. Virtual sensing, sensor recovery, fault localization, imputation, and control-oriented reasoning all require identifying Points related to a target through building structure, and evaluating topology-routed context there is left to future work.

6 Conclusion

This work proposes TopoBrick, a training-free framework for zero-shot building IoT forecasting. It samples physically coupled exogenous variables for each target by agentic reasoning over the building knowledge graph and organizes them by deployment-time availability, letting a frozen foundation model condition on meaningful context without building-specific training. On three real-world buildings TopoBrick outperforms zero-shot foundation models in most cases and remains competitive with fully trained building-specific forecasters, with benefits that depend on how closely the routed variables match the building’s physical coupling. Building knowledge graphs can therefore serve as effective routing layers for forecasting across heterogeneous cyber-physical environments.

Acknowledgment

This research is funded by the NSW Government through CSIRO’s NSW Digital Infrastructure Energy Flexibility (DIEF) project as part of the Net Zero Plan Stage 1: 2020-2030, and by the Reliable Affordable Clean Energy for 2030 (RACE for 2030) Cooperative Research Centre.

References

- [1] Abdul Fatir Ansari, Oleksandr Shchur, Jaris Küken, Andreas Auer, Boran Han, Pedro Mercado, Syama Sundar Rangapuram, Huibin Shen, Lorenzo Stella, Xiyuan Zhang, et al. 2025. Chronos-2: From Univariate to Universal Forecasting. *arXiv preprint arXiv:2510.15821* (2025).
- [2] Abdul Fatir Ansari, Lorenzo Stella, Caner Turkmen, Xiyuan Zhang, Pedro Mercado, Huibin Shen, Oleksandr Shchur, Syama Sundar Rangapuram, Sebastian Pineda Arango, Shubham Kapoor, et al. 2024. Chronos: Learning the language of time series. *arXiv preprint arXiv:2403.07815* (2024).
- [3] Bharathan Balaji, Arka Bhattacharya, Gabriel Fierro, Jingkun Gao, Joshua Gluck, Dezhi Hong, Aslak Johansen, Jason Koh, Joern Ploennigs, Yuvraj Agarwal, et al. 2016. Brick: Towards a unified metadata schema for buildings. In *Proceedings of the 3rd ACM International Conference on Systems for Energy-Efficient Built Environments*. 41–50.
- [4] Abhimanyu Das, Weihao Kong, Rajat Sen, and Yichen Zhou. 2023. A decoder-only foundation model for time-series forecasting. *arXiv preprint arXiv:2310.10688* (2023).
- [5] Jiawei Hu, Hong Jia, Mahbub Hassan, Lina Yao, Brano Kusy, and Wen Hu. 2025. Lightllm: A versatile large language model for predictive light sensing. In *Proceedings of the 23rd ACM Conference on Embedded Networked Sensor Systems*. 158–171.
- [6] Cong Huang, Jack CP Cheng, Fangli Hou, Zhaoji Wu, and Helen HL Kwok. 2026. Reliable Indoor Air Quality Prediction with Uncertainty-Aware Spatiotemporal GNN. In *Proceedings of the 13th ACM International Conference on Systems for Energy-Efficient Buildings, Cities, and Transportation*. 324–333.
- [7] Furong Jia, Kevin Wang, Yixiang Zheng, Defu Cao, and Yan Liu. 2024. Gpt4mts: Prompt-based large language model for multimodal time-series forecasting. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 23343–23351.
- [8] Ming Jin, Shiyu Wang, Lintao Ma, Zhixuan Chu, James Y Zhang, Xiaoming Shi, Pin-Yu Chen, Yuxuan Liang, Yuan-Fang Li, Shirui Pan, and Qingsong Wen. 2024. Time-LLM: Time series forecasting by reprogramming large language models. In *International Conference on Learning Representations (ICLR)*.
- [9] Yun-Dam Ko, Hyeyun Eunice Jung, Marco Pritoni, and Rishee Jain. 2026. BrickQA: Bridging the Semantic Gap in Building Operations with Dynamic Graph Exploration. In *Proceedings of the 13th ACM International Conference on Systems for Energy-Efficient Buildings, Cities, and Transportation*. 207–217.
- [10] Sungmin Lee, Joonhee Lee, Minju Kang, Seungyong Lee, Dongju Kim, Jingi Hong, Jun Shin, Pei Zhang, and JeongGil Ko. 2026. JARVIS for HVAC: LLM-based Question-Answer Framework for Sensor-driven HVAC System Interaction. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies* 10, 2 (2026), 1–36.
- [11] Xiachong Lin, Arian Prabowo, Imran Razzak, Hao Xue, Matthew Amos, Sam Behrens, and Flora D Salim. 2024. Bitsa: Leveraging time series foundation model for building energy analytics. In *2024 IEEE International Conference on Data Mining Workshops (ICDMW)*. IEEE, 891–894.
- [12] Chenghao Liu, Taha Aksu, Juncheng Liu, Xu Liu, Hanshu Yan, Quang Pham, Silvio Savarese, Doyen Sahoo, Caiming Xiong, and Junnan Li. 2025. Moirai 2.0: When less is more for time series forecasting. *arXiv preprint arXiv:2511.11698* (2025).
- [13] Yong Liu, Tengge Hu, Haoran Zhang, Haixu Wu, Shiyu Wang, Lintao Ma, and Mingsheng Long. 2024. itransformer: Inverted transformers are effective for time series forecasting. In *International conference on learning representations*, Vol. 2024. 11116–11140.
- [14] Yong Liu, Guo Qin, Xiangdong Huang, Jianmin Wang, and Mingsheng Long. 2024. Autotimes: Autoregressive time series forecasters via large language models. *Advances in Neural Information Processing Systems* 37 (2024), 122154–122184.
- [15] Na Luo, Zhe Wang, David Blum, Christopher Weyandt, Norman Bourassa, Mary Ann Piette, and Tianzhen Hong. 2022. A three-year dataset supporting research on building energy management and occupancy analytics. *Scientific data* 9, 1 (2022), 156.
- [16] Aitian Ma, Dongsheng Luo, and Mo Sha. 2026. MixLinear: Extreme Low Resource Multivariate Time Series Forecasting with 0.1K Parameters. In *International Conference on Learning Representations*, Vol. 2026. 77764–77789.
- [17] Ozan Baris Mulayim, Avia Anwar, Umut Mete Saka, Lazlo Paul, Anand Krishnan Prakash, Gabe Fierro, Marco Pritoni, and Mario Bergés. 2025. BuildingQA: A Benchmark for Natural Language Question Answering over Building Knowledge Graphs. In *Proceedings of the 12th ACM International Conference on Systems for Energy-Efficient Buildings, Cities, and Transportation*. 65–75.
- [18] Ozan Baris Mulayim, Gabe Fierro, Mario Bergés, and Marco Pritoni. 2025. Towards Zero-shot Question Answering in CPS-IoT: Large Language Models and Knowledge Graphs. In *Proceedings of the 2nd International Workshop on Foundation Models for Cyber-Physical Systems & Internet of Things*. 7–12.
- [19] Yuqi Nie, Nam H Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. 2022. A time series is worth 64 words: Long-term forecasting with transformers. *arXiv preprint arXiv:2211.14730* (2022).
- [20] Arian Prabowo, Kaixuan Chen, Hao Xue, Subbu Sethuvenkatraman, and Flora D Salim. 2023. Navigating Out-of-Distribution Electricity Load Forecasting during COVID-19: Benchmarking energy load forecasting models without and with continual learning. In *Proceedings of the 10th ACM international conference on systems for energy-efficient buildings, cities, and transportation*. 41–50.
- [21] Arian Prabowo, Xiachong Lin, Imran Razzak, Hao Xue, Matthew Amos, Stephen D White, and Flora D Salim. 2025. Brick-by-Brick: Cyber-Physical Building Data Classification Challenge. In *Companion Proceedings of the ACM on Web Conference 2025*. 3021–3025.
- [22] Arian Prabowo, Xiachong Lin, Imran Razzak, Hao Xue, Emily W Yap, Matthew Amos, and Flora D Salim. 2024. Building Timeseries Dataset: Empowering Large-Scale Building Analytics. *Advances in Neural Information Processing Systems* 37 (2024), 133180–133206.
- [23] Shiyu Wang, Haixu Wu, Xiaoming Shi, Tengge Hu, Huakun Luo, Lintao Ma, James Zhang, and Jun Zhou. 2024. Timemixer: Decomposable multiscale mixing for time series forecasting. In *International conference on learning representations*, Vol. 2024. 38626–38652.
- [24] Yuxuan Wang, Haixu Wu, Jiaxiang Dong, Guo Qin, Haoran Zhang, Yong Liu, Yunzhong Qiu, Jianmin Wang, and Mingsheng Long. 2024. Timexer: Empowering transformers for time series forecasting with exogenous variables. *Advances in Neural Information Processing Systems* 37 (2024), 469–498.
- [25] Gerald Woo, Chenghao Liu, Akshat Kumar, Caiming Xiong, Silvio Savarese, and Doyen Sahoo. 2024. Unified training of universal time series forecasting transformers. In *Forty-first International Conference on Machine Learning*.
- [26] Mingyuan Xia, Chunxu Zhang, Zijian Zhang, Hao Miao, Qidong Liu, Yuan-shao Zhu, and Bo Yang. 2025. TimeEmb: A Lightweight Static-Dynamic Disentanglement Framework for Time Series Forecasting. In *NeurIPS*. <https://openreview.net/forum?id=sLfmvrk6T>
- [27] Lei Xu, Maomao Hu, and Cheng Fan. 2022. Probabilistic electrical load forecasting for buildings using Bayesian deep neural networks. *Journal of Building Engineering* 46 (2022), 103853.
- [28] Zhijian Xu, Ailing Zeng, and Qiang Xu. 2024. FITS: Modeling time series with 10k parameters. In *International Conference on Learning Representations*, Vol. 2024. 26295–26318.
- [29] Hao Xue and Flora D Salim. 2023. Promptcast: A new prompt-based learning paradigm for time series forecasting. *IEEE Transactions on Knowledge and Data Engineering* 36, 11 (2023), 6851–6864.
- [30] Du Yin, Jinliang Deng, Shuang Ao, Zechen Li, Hao Xue, Arian Prabowo, Renhe Jiang, Xuan Song, and Flora Salim. 2024. Enhancing spatio-temporal quantile forecasting with curriculum learning: Lessons learned. In *Proceedings of the 32nd ACM International Conference on Advances in Geographic Information Systems*. 42–53.
- [31] Du Yin, Hao Xue, Arian Prabowo, Shuang Ao, and Flora Salim. 2025. XXLTraffic: Expanding and Extremely Long Traffic forecasting beyond test adaptation. In *Proceedings of the 33rd ACM International Conference on Advances in Geographic Information Systems*. 511–521.
- [32] Zakia Zaman, Praveen Gauravaram, Sanjay Jha, and Wen Hu. 2025. Bloom-LLM: Privacy-Preserving Large Language Model for Load Forecasting. In *Proceedings of the 12th ACM International Conference on Systems for Energy-Efficient Buildings, Cities, and Transportation*. 128–138.
- [33] Ailing Zeng, Muxi Chen, Lei Zhang, and Qiang Xu. 2023. Are transformers effective for time series forecasting?. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 37. 11121–11128.
- [34] Zijian Zhang, Ze Huang, Zhiwei Hu, Xiangyu Zhao, Wanyu Wang, Zitao Liu, Junbo Zhang, S Joe Qin, and Hongwei Zhao. 2023. Mlpst: Mlp is all you need for spatio-temporal prediction. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*. 3381–3390.
- [35] Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang. 2021. Informer: Beyond efficient transformer for long sequence time-series forecasting. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 35. 11106–11115.
- [36] Tian Zhou, Peisong Niu, Liang Sun, Rong Jin, et al. 2023. One fits all: Power general time series analysis by pretrained lm. *Advances in neural information processing systems* 36 (2023), 43322–43355.

A Preprocessing and Forecast Target Construction

We design a four-stage pre-processing procedure to separate the failure modes that would be conflated by a single filter, avoiding the discard of a sensor with sparse local corruption.

L1: Observation-level range filtering. Each observation (i, t) carries a quality-control mask $m_{i,t}^{qc} \in \{0, 1\}$. A reading is masked, without removing its sensor, when it is NaN or infinite, matches a known sentinel code, exceeds an overflow threshold of 10^9 , falls outside the physically plausible range for its Brick class, or is flagged as an abnormal drop to zero, a frozen run, or a robust MAD outlier. Temperature-like sensors are canonicalized to degrees Celsius before their bounds are applied.

L2: Sensor-level usability auditing. Each sensor then receives a usability flag u_i , and the exogenous pool is $\mathcal{U}_{\text{exo}} = \{i \mid u_i = 1\}$. A sensor leaves the pool when its cleaned signal is still statistically unreliable, which covers post-QC coverage below 0.4, near-flat dynamics, almost-all-zero measurements, and monotonic cumulative-meter behaviour. Where L1 masks readings, L2 removes sensors.

L3: Semantic and empirical target filtering. A sensor is kept as a forecast target when $M_i^{\text{target}} = M_i^{\text{meas}} \wedge M_i^{\text{eligible}} \wedge u_i$, where M_i^{meas} marks it as a measurement Point and M_i^{eligible} excludes classes unsuited to continuous prediction: control and configuration variables, raw electrical identifiers, actuators, counters, setpoints, status and command points, and unmapped entities.

L4: Window-level validation. Sensors whose training-split variation falls below $\text{std}_{\text{train}} = 0.1$ are dropped, and a window ending at t is kept only when $\frac{1}{H} \sum_{\tau=t+1}^{t+H} m_{i,\tau}^{qc} \geq 0.95$. This removes outage-dominated windows while keeping sensors that are usable in other periods.

B Baseline Implementation Details

All supervised baselines use Adam for up to 30 epochs with masked MSE on the per-sensor normalized scale, gradient clipping at $\|\cdot\|_2 \leq 1$, and early stopping with patience 5. Invalid multivariate windows are zero-filled and excluded from loss and evaluation. For multivariate settings, absent channels are additionally masked as attention keys to prevent duplicate zero tokens from absorbing softmax mass. All other settings remain unchanged.

C Pseudocode

Algorithm 1 shows the pipeline pseudocode for Section 3.1 and Section 3.2. Each target uses two agent calls: sampling and in-place verification. Verifier removals precede additions, after which the selection is capped at $n_{\text{max}} = 60$. The cap applies to 25.6% of BTS-C targets and at most 2% in the other buildings.

D Prompts and Output Examples for the Agents

Prompts for Topology Sampler. The box below gives the system prompt used by the topology sampler.

Algorithm 1 Agentic topology subgraph sampling.

Require: target Point p ; skeleton $G_s = (V_s, E_s)$; size cap n_{max}

Ensure: exogenous-variable set $\mathcal{E}(p)$ and its subgraph $G(p)$

Phase 1: deterministic context construction

- 1: $A \leftarrow \{u \in V_s \mid (u, \text{hasPoint}, p) \in E\}$ ▷ target anchors
- 2: **if** $A = \emptyset$ **then** ▷ p has no skeleton anchor
- 3: $U \leftarrow$ recovered equipment cluster of p
- 4: **else**
- 5: $U \leftarrow \bigcup_{a \in A} \text{UPSTREAM}(a, G_s)$ ▷ to the top of G_s
- 6: **end if**
- 7: $(C(p), \alpha) \leftarrow \text{CONTEXT}(p, U, G_s)$

Phase 2: topology sampler – one agent call

- 8: $\mathcal{A}(p) \leftarrow \text{SAMPLE}(C(p))$ ▷ actions (a_i, ω_i, q_i)
- 9: $\mathcal{L}(p) \leftarrow \text{MATERIALIZE}(\mathcal{A}(p), \alpha)$ ▷ deduplicated, p excluded

Phase 3: physics-consistency verifier – one agent call

- 10: **if** the verifier is enabled **then**
- 11: $(\mathcal{D}, \mathcal{A}^+) \leftarrow \text{VERIFY}(C(p), \text{classes}(\mathcal{L}(p)))$
- 12: $\mathcal{L}(p) \leftarrow [\ell \in \mathcal{L}(p) \mid \text{class}(\ell) \notin \mathcal{D}]$ ▷ removals first
- 13: append $\text{MATERIALIZE}(\mathcal{A}^+, \alpha) \setminus \mathcal{L}(p)$ to $\mathcal{L}(p)$
- 14: **end if**

Phase 4: deterministic assembly

- 15: $\mathcal{E}(p) \leftarrow \mathcal{L}(p)[1:n_{\text{max}}]$
- 16: $G(p) \leftarrow$ subgraph on $U \cup \{p\} \cup \mathcal{E}(p)$, one hasPoint edge per selected Point
- 17: **return** $\mathcal{E}(p), G(p)$

Sampler System Prompt

You are a building-systems engineer choosing covariate time series to help a forecasting model predict a target sensor. The target can be any kind of
 ↳ point: temperature, electrical power, energy, pressure, flow, humidity,
 ↳ status, or another Brick point class. Reason about this target's own
 ↳ physics.

Reason explicitly before answering, then output exactly one fenced JSON
 ↳ block.

Selection rubric:

1. Target physics. Identify what the target measures and what physically
 ↳ drives that quantity. Drivers are domain-specific. For example:
 ↳ temperature: upstream supply, return, or mixed-air temperatures,
 ↳ setpoints, and weather;
 ↳ electrical power/current/energy: voltage, current, metered load, and
 ↳ building demand;
 ↳ flow/pressure: upstream fan, pump, valve, and the loop it sits on;
 ↳ humidity/CO2: ventilation, occupancy, and outdoor humidity.
2. Topology localization. Read the target node, its serving or containing
 ↳ equipment or group, and its upstream chain.
3. Co-located state. Include relevant points directly attached to the
 ↳ target's own node, such as setpoints, controls, statuses, or co-located
 ↳ sensors.
4. Same-class peer cohort. Choose the tightest physically meaningful anchor,
 ↳ such as the immediate serving equipment, meter, circuit, or zone group.
 ↳ Prefer a small same-serving-equipment cohort over a building-wide
 ↳ same-class dump. Use building-wide expansion only when the local cohort
 ↳ is too small.
5. Domain drivers. Pull the drivers identified in Step 1 from the own node,
 ↳ upstream chain, or global driver hosts. Do not pull variables that are
 ↳ not physically coupled to the target's domain.

Self-check:

- Include the target's own controls or co-located points when relevant.
- Include a tight same-class peer cohort rather than a building-wide flood.
- Include the domain drivers identified above when they are available.
- Reject off-domain variables with no coupling pathway, static configuration
 ↳ points, and duplicates.

Context format:

- YOU ARE HERE contains the target node and its ancestry to the root. feeds
 ↳ denotes upstream flow or supply, including the circuit that powers it.
 ↳ hasPart and isLocationOf denote spatial containment.
 - NEIGHBORHOOD contains anchors [A#], their directly attached point classes,
 ↳ and their direct child equipment or zones.
 - GLOBAL DRIVERS contains building-wide hosts such as weather stations,
 ↳ utility meters, central plant, and loops.

Each pull action has the form:

```
{
  "anchor": "A#",
  "expand": "self" | "subtree" | "building" | "cluster",
  "point_class": "<Brick point class>",
  "reason": "short rationale"
}
```

Expansion semantics:

- self: points of the requested class directly attached to the anchor.
- subtree: points of the requested class under the anchor subtree.
- building: points of the requested class across the entire building. Use
 ↳ only for scarce global signals, not common classes that appear on many
 ↳ unrelated equipment.
- cluster: recovered cluster mates for KG-orphan targets.

Return exactly:

```
{
  "pulls": [
    {
      "anchor": "A#",
      "expand": "self" | "subtree" | "building" | "cluster",
      "point_class": "<Brick point class>",
      "reason": "..."
    }
  ],
  "summary": "one-line summary"
}
```

Prompts for Verifier. The box below gives the system prompt used by the physics-consistency verifier.

Verifier System Prompt

You are an independent senior building-systems engineer auditing another
 ↳ engineer's covariate selection for a forecasting target. You are shown
 ↳ the same building neighborhood and the selected covariate set summarized
 ↳ by Brick class and count.

Audit only for this target's physics:

1. Peer cohort. Check whether same-class spatial peers from the target's
 ↳ serving equipment or group are included. If such peers clearly exist
 ↳ nearby but are missing or under-represented, flag this as a gap.
2. Domain drivers. Check whether available physical drivers of the target
 ↳ quantity are present: weather for thermal variables; voltage/current for
 ↳ power; upstream fan, pump, valve, or flow variables for pressure/flow;
 ↳ and ventilation or occupancy variables for humidity/CO2.
3. Noise. Check whether the selection contains off-domain classes without a
 ↳ plausible coupling pathway, or an excessively broad same-class dump that
 ↳ should be trimmed.

Most selections are already acceptable. Flag only real gaps or excesses, and
 ↳ only use variables actually available in the shown neighborhood.

Return exactly one JSON object:

```
{
  "verdict": "ok" | "revise",
  "add": [
    {
      "anchor": "A#",
      "expand": "self" | "subtree" | "building" | "cluster",
      "point_class": "...",
      "reason": "missing peer" | "missing driver"
    }
  ],
  "drop": ["BrickClass_to_remove", "..."],
  "note": "one-line summary"
}
```

If the selection is already good, return:

```
{ "verdict": "ok", "add": [], "drop": [], "note": "..." }
```

Sampled Output. The box below shows the sampler output for a zone air temperature target (zone_016_temp, LBNL59), reproduced verbatim from the released subgraph artifact. The upper block is

what the agent returns and conforms to the schema of the prompt above. The lower block lists the fields that the deterministic materializer and the verifier attach afterwards, which are recorded alongside it but are not part of the agent's response.

Sampler Output (JSON)

```
### agent response (conforms to the schema in the prompt above)
{
  "pulls": [
    { "anchor": "A1", "expand": "self",
      "point_class": "Heating_Temperature_Setpoint",
      "reason": "Direct control of zone heating setpoint"},
    { "anchor": "A1", "expand": "self",
      "point_class": "Cooling_Temperature_Setpoint",
      "reason": "Direct control of zone cooling setpoint"},
    { "anchor": "A1", "expand": "self",
      "point_class": "Supply_Air_Flow_Sensor",
      "reason": "Supply flow directly influences zone temperature"},
    { "anchor": "A3", "expand": "subtree",
      "point_class": "Supply_Air_Temperature_Sensor",
      "reason": "Upstream supply air temperature drives zone temperature"},
    { "anchor": "A3", "expand": "subtree",
      "point_class": "Mixed_Air_Temperature_Sensor",
      "reason": "Mixed air temperature is a key upstream driver"},
    { "anchor": "A4", "expand": "subtree",
      "point_class": "Zone_Air_Temperature_Sensor",
      "reason": "Peer cohort of zone temps within the same HVAC zone"},
    { "anchor": "A5", "expand": "self",
      "point_class": "Occupant_Count",
      "reason": "Occupancy load influences zone temperature"},
    { "anchor": "A6", "expand": "building",
      "point_class": "Outdoor_Air_Temperature_Sensor",
      "reason": "Outdoor temperature is a global driver for zone thermal
        ↳ balance"}
  ],
  "summary": "Pull zone setpoints, flow, upstream air temps, peer zone temps,
    ↳ occupancy, and outdoor temp"
}

### attached downstream (materializer, then verifier)
n_resolved per pull: 1, 1, 1, 1, 1, 10, 1, 2
verifier: { "verdict": "ok", "added": 0, "dropped_classes": [],
  "note": "All peer, driver, and noise checks satisfied." }
subgraph: 25 nodes, 14 edges, spine_size 6
leaves: 18 (n_pre_cap 18, below the 60-leaf cap)
roots_reached: Building, RTU01
```

E Cost of Subgraph Construction

Each target requires two LLM calls, one for sampling and one for verification. Subgraphs are constructed once during building onboarding and reused across all prediction horizons. Prompt and completion denote API-reported input and output token usage, respectively.

Table 5: Token usage for subgraph construction

Building	Prompt	Completion	Total
LBNL59	2,412	1,353	3,765
BTS-B	4,718	1,092	5,810
BTS-C	12,165	995	13,160

Variation in token usage is driven primarily by prompt length. Larger topology contexts increase input usage, whereas completion usage remains comparatively stable. Because subgraphs are constructed once and reused, this overhead is incurred once per target rather than per forecast. Decoding is nondeterministic, so the accounting pass may not reproduce the released subgraphs exactly.